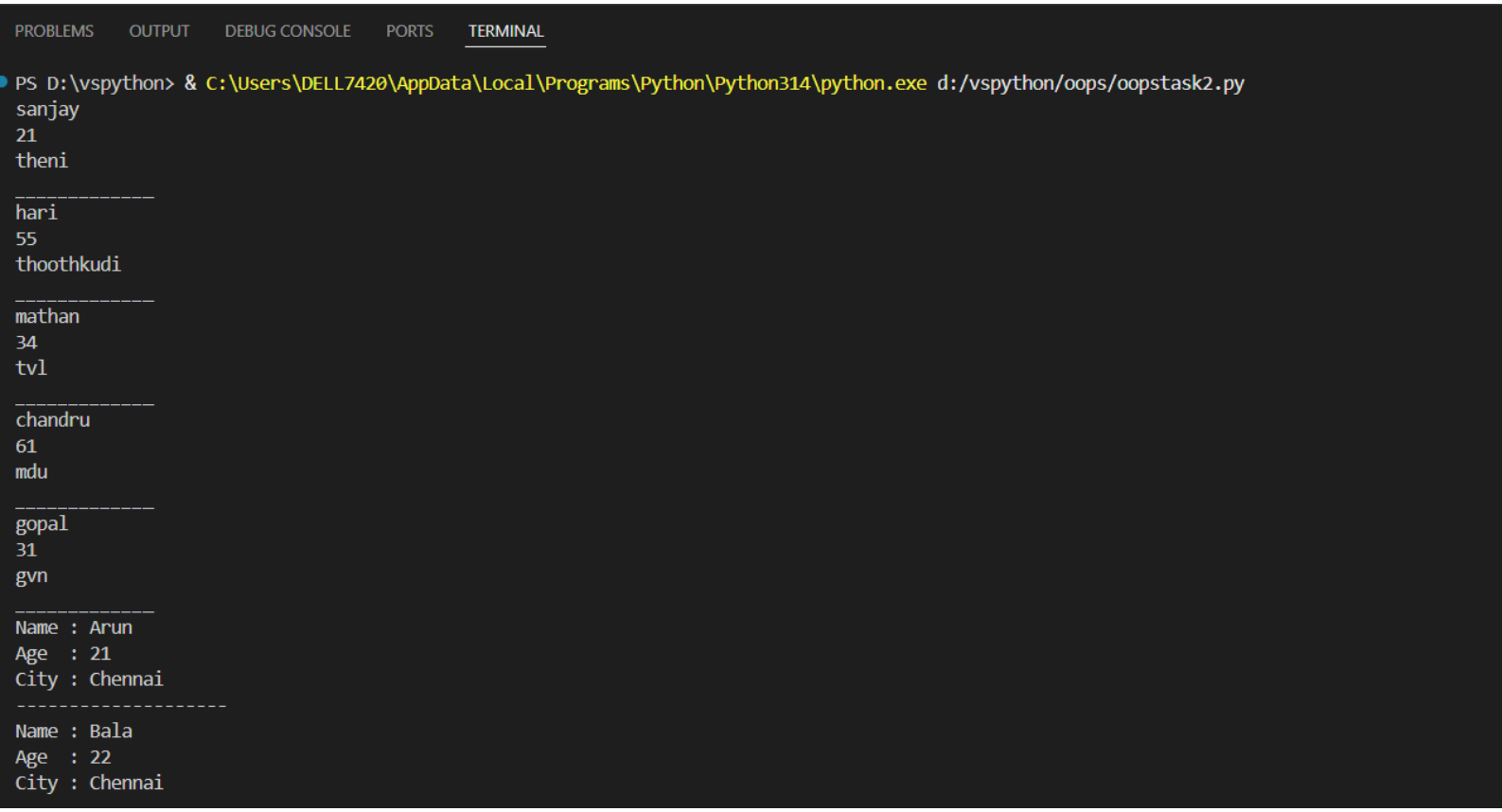
oopstask2.py > ...

```python
# create a class with instance variable of name,ae,city and pass instance methods for the same with 5 object creation
class student:
    def __init__(self,name,age,city):
        self.n1=name
        self.a1=age
        self.c1=city
    def display(self):
      print(self.n1)
      print(self.a1)
      print(self.c1)
      print("_____________")
person1=student("sanjay",21,"theni")
person2=student("hari",55,"thoothkudi")
person3=student("mathan",34,"tvl")
person4=student("chandru",61,"mdu")
person5=student("gopal",31,"gvn")
person1.display()
person2.display()
person3.display()
person4.display()
person5.display()
```

```python

#Access a class variable of city='chennai' and pass same value for all objects.
#for this pass 5object value
class Student:
    city = "Chennai"
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print("Name :", self.name)
        print("Age  :", self.age)
        print("City :", Student.city)
        print("--------------------")
s1 = Student("Arun", 21)
s2 = Student("Bala", 22)
s3 = Student("Charan", 20)
s4 = Student("Deepak", 23)
s5 = Student("Eswar", 21)
s1.display()
s2.display()
s3.display()
s4.display()
s5.display()
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
PORTS
TERMINAL
PS D:\vspython> & C:\Users\DELL7420\AppData\Local\Programs\Python\Python314\python.exe d:/vspython/oops/oopstask2.py
sanjay
21
theni
_____
hari
55
thoothkudi
_____
mathan
34
tvl
_____
chandru
61
mdu
_____
gopal
31
gvn
_____
Name : Arun
Age : 21
City : Chennai

Name : Bala
Age : 22
City : Chennai

```
------------
Name : Arun
Age  : 21
City : Chennai
--------------------
Name : Bala
Age  : 22
City : Chennai
--------------------
Name : Charan
Age  : 20
City : Chennai
--------------------
Name : Deepak
Age  : 23
City : Chennai
--------------------
Name : Eswar
Age  : 21
City : Chennai
--------------------
PS D:\vspython>
```

What is OOps in python?

⇒ OOps in python is a programming approch in which data (attributes) and function (methods) are combined into objects created form class.

→ (Object oriented programming system)

is a programming paradigm that organizes code using class and object. It ~~makes~~ helps makes programs more organized, reusable, secure, and easy to maintain.

Main Components in OOps

Class - A blueprint for creating objects

object - A instance of class

Attributes - Variable that store in object data

methods - Function define inside a class

Constructor (__init__) - A special method that initialize object data

Self - It is reference to current object access its own data of class. It allow each object to store ad

Encapsulation : Bundles data and method together and control access

Inheritance : Allows one class to inherit properties and method from another class

polymorphism : The same method can behave differently for different object

Abstraction : Hides implementation details and show only essential features

Components of Python

variable - Store values

Datatype - Define type of data (int, float, str, list)

operators - (+, -, *, /, ==)

Input/output - Read input and display output (input(), print())

Conditional Stmt - Decision making (if, else, elif)

Loops - for, while

Function - Reusable block of code

Modules - Python files containing reusable code

Package - Collection of related modules

Exceptions - Handle runtime errors (try, except)

File handling - Read from and write to files.

OOps - Organize programs using class and object

Types of Inheritance

Single inheritance

multiple inheritance

multilevel inheritance

Hierarchical inheritance

Hybrid inheritance.

Uses of __init__

→ It is special method is called Constructor in Python. It automatically executed when object is created.

(or)

__init__() is constructor that automatically run when object is created. It used to initialize the object data.

Scenario for Inheritance

Inheritance is useful when multiple classes share common properties and behaviours.

Real Time Example :

```
Class person :
    def display (self)
        print ("person Details")

Class student (person) :
    def study (self):
        print ("student is studying")

S1 = student ()
S1 . display
S1 . study .
```